

Modelo Predictivo de Turismo Internacional — México

XGBoost

Objetivo del proyecto

Predecir el flujo mensual de turistas extranjeros que arriban a México por aeropuerto, país de residencia y género. Los datos provienen del sistema **DATATUR** de la Secretaría de Turismo de México, período 2012–2025.

Algoritmo: XGBoost (Extreme Gradient Boosting)

A diferencia de Random Forest, que construye muchos árboles independientes y promedia sus predicciones, XGBoost construye los árboles en secuencia: cada árbol nuevo analiza dónde se equivocó el árbol anterior y aprende a corregir exactamente esos errores. Este proceso se llama **gradient boosting**.

La ventaja es que el modelo enfoca toda su capacidad en los casos más difíciles de predecir. La desventaja es que requiere más cuidado en su configuración para evitar que memorice el ruido de los datos de entrenamiento.

Estrategia de entrenamiento en dos fases

XGBoost tiene un mecanismo llamado **early stopping**: monitorea el error en un conjunto de validación mientras entrena y se detiene automáticamente cuando ese error deja de mejorar. Esto evita el sobreajuste sin tener que adivinar cuántos árboles son suficientes.

Para que este mecanismo sea válido, el conjunto de validación debe ser **posterior en el tiempo** al conjunto de entrenamiento. Si se usara datos mezclados, el modelo tendría señales del futuro y sus resultados serían artificialmente buenos.

FASE 1 – Calibración:

Entrenamiento : 2012 – Junio 2022

Early stopping: Julio – Diciembre 2022

Resultado : número óptimo de árboles (BEST_N)

FASE 2 – Modelo final:

Entrenamiento : 2012 – 2022 completo

Árboles : exactamente BEST_N

Evaluación : 2023, 2024, 2025

Usar 2022 completo en la Fase 2 (en lugar de solo hasta junio) añade más datos de entrenamiento sin comprometer la honestidad de la evaluación, ya que ningún año de validación (2023-2025) aparece en el

Umbrales de evaluación

Umbral	Interpretación
≥ 250 turistas	Actividad mínima para planificación
$\geq 1,000$ turistas	Rutas comercialmente activas
$\geq 10,000$ turistas	Destinos de alta demanda
$\geq 100,000$ turistas	Segmentos premium en temporada alta

Activar GPU antes de ejecutar: Entorno de ejecución → Cambiar tipo de entorno → T4 GPU

1. Configuración del entorno

Se importan las bibliotecas necesarias y se verifica si hay una GPU disponible. XGBoost puede usar CUDA (GPU de NVIDIA) para acelerar el entrenamiento hasta 10 veces comparado con CPU. Si no hay GPU, el modelo corre en CPU de todas formas pero tarda más.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import pickle, json, time, os
import xgboost as xgb
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

plt.rcParams['figure.figsize'] = (13, 5)
plt.rcParams['axes.spines.top'] = False
plt.rcParams['axes.spines.right'] = False

print(f'XGBoost : {xgb.__version__}')
print(f'pandas : {pd.__version__}')

try:
    _t = xgb.XGBRegressor(device='cuda', n_estimators=3)
    _t.fit(np.random.rand(10, 3), np.random.rand(10))
    print('GPU CUDA: disponible – entrenamiento acelerado')
    DEVICE = 'cuda'
except Exception as e:
    print(f'GPU no disponible ({e})')
    print('Usando CPU – activar GPU en Entorno de ejecución → T4 GPU')
    DEVICE = 'cpu'
```

```
XGBoost : 3.2.0
pandas : 2.2.2
GPU CUDA: disponible – entrenamiento acelerado
```

2. Carga del dataset

El dataset ya fue limpiado y procesado en etapas anteriores. Cada fila es una combinación única de aeropuerto, país de residencia del turista, género, mes y año.

Las columnas de rezagos (`Lag_1_mes` , `Lag_12_meses` , `Rolling_3m`) fueron pre-calculadas: representan el turismo del mes anterior, el mismo mes del año pasado, y el promedio de los últimos tres meses.

```
In [2]: URL = 'https://raw.githubusercontent.com/Montiel-Oscar/predictive-tourism-intelligence-mexico/re

print('Cargando dataset...')
df = pd.read_csv(URL)
if 'Año' in df.columns:
    df = df.rename(columns={'Año': 'Ano'})

TARGET = 'Valor_Residencia'
MESES = ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun', 'Jul', 'Ago', 'Sep', 'Oct', 'Nov', 'Dic']
UMBRALES = [250, 1000, 10000, 100000]

print(f'Filas cargadas      : {df.shape[0],}')
print(f'Columnas           : {df.shape[1],}')
print(f'Años disponibles    : {sorted(df["Ano"].unique())}')
print(f'Aeropuertos únicos : {df["Aeropuerto"].nunique()}')
print(f'Países de origen    : {df["Pais"].nunique()}')
```

Cargando dataset...

Filas cargadas : 371,011

Columnas : 23

Años disponibles : [np.int64(2012), np.int64(2013), np.int64(2014), np.int64(2015), np.int64(2016), np.int64(2017), np.int64(2018), np.int64(2019), np.int64(2022), np.int64(2023), np.int64(2024), np.int64(2025)]

Aeropuertos únicos : 66

Países de origen : 243

3. Ingeniería de variables

Se construyen cinco variables adicionales que complementan los rezagos temporales del dataset base:

mes_share_aeropuerto — Estacionalidad estructural: Para cada aeropuerto, el porcentaje de su tráfico anual que llega en cada mes. Huatulco concentra el 60% de sus turistas entre enero y marzo, casi nada en verano. Cancún tiene una distribución más uniforme pero con picos en Navidad. Esta variable se calcula únicamente sobre datos hasta junio 2022 para no filtrar información del futuro hacia el entrenamiento.

log_lag1, log_rolling, log_lag12 — Rezagos logarítmicos: El modelo aprende sobre el logaritmo del target porque el flujo de turistas es muy asimétrico (entre 1 y 300,000). Tener los rezagos en la misma escala logarítmica facilita que el modelo aprenda las relaciones multiplicativas directamente.

ratio_lag1_lag12 — Crecimiento interanual: Compara el último mes con el mismo mes del año pasado. Un valor mayor a 1 indica que el turismo está creciendo.

tendencia_reciente — Momentum de corto plazo: Compara el último mes con el promedio de los tres meses anteriores. Detecta si hay una aceleración o desaceleración reciente.

```
In [3]: # Estacionalidad histórica por aeropuerto
# Calculada solo sobre datos hasta junio 2022 (sin filtración temporal)
df_share = df[
    (df['Ano'] < 2022) | ((df['Ano'] == 2022) & (df['MesNum'] <= 6))
].copy()
anual = (
```

```

df_share.groupby(['Aeropuerto', 'Ano'])[TARGET]
    .sum().reset_index().rename(columns={TARGET: 'Total_anual'})
)
mes_a = (
    df_share.groupby(['Aeropuerto', 'Ano', 'MesNum'])[TARGET]
    .sum().reset_index().merge(anual, on=['Aeropuerto', 'Ano'])
)
mes_a['share'] = mes_a[TARGET] / mes_a['Total_anual'].clip(lower=1)
share_m = (
    mes_a.groupby(['Aeropuerto', 'MesNum'])['share']
    .mean().reset_index().rename(columns={'share': 'mes_share_aeropuerto'})
)
df = df.merge(share_m, on=['Aeropuerto', 'MesNum'], how='left')
df['mes_share_aeropuerto'] = df['mes_share_aeropuerto'].fillna(1/12)

# Rezagos en escala Logarítmica
df['log_lag1'] = np.log1p(df['Lag_1_mes'])
df['log_rolling'] = np.log1p(df['Rolling_3m'])
df['log_lag12'] = np.log1p(df['Lag_12_meses'])

# Ratios de tendencia (acotados para evitar valores extremos)
eps = 1.0
df['ratio_lag1_lag12'] = (df['Lag_1_mes'] / (df['Lag_12_meses'] + eps)).clip(0.2, 5.0)
df['tendencia_reciente'] = (df['Lag_1_mes'] / (df['Rolling_3m'] + eps)).clip(0.2, 5.0)

if 'Años_desde_inicio' not in df.columns and 'Anos_desde_inicio' in df.columns:
    df = df.rename(columns={'Anos_desde_inicio': 'Años_desde_inicio'})

print('Variables adicionales creadas correctamente.')
print(f'Nulos nuevos: {df[["mes_share_aeropuerto", "log_lag1", "ratio_lag1_lag12"]].isnull().sum()}')

```

Variables adicionales creadas correctamente.

Nulos nuevos: 0

4. División temporal y definición de variables predictoras

Se definen los cuatro conjuntos de datos necesarios para la estrategia de dos fases.

El punto crítico es que **ningún dato de 2023 en adelante aparece en ningún conjunto de entrenamiento**. Los tres `assert` al final verifican automáticamente esta condición y detienen el notebook con un error claro si algo está mal.

A diferencia de Random Forest, XGBoost recibe el target en escala logarítmica (`log1p`) en la Fase 1 porque su función de pérdida estándar (error cuadrático) funciona mejor en esa escala para distribuciones asimétricas.

```

In [4]: FEATURES = [
    # Posición en el calendario
    'MesNum', 'mes_sin', 'mes_cos', 'Trimestre',
    'Es_temporada_alta', 'Años_desde_inicio',
    # Estacionalidad histórica del aeropuerto
    'mes_share_aeropuerto',
    # Identificadores codificados numéricamente
    'Sexo_enc', 'Region_enc', 'Aeropuerto_enc', 'Pais_enc', 'nac_imputada',
    # Rezagos en escala original
    'Lag_1_mes', 'Lag_2_mes', 'Lag_3_mes', 'Lag_12_meses', 'Rolling_3m',
    # Rezagos en escala Logarítmica

```



```

    'log_lag1', 'log_rolling', 'log_lag12',
    # Indicadores de tendencia
    'ratio_lag1_lag12', 'tendencia_reciente'
]
FEATURES = [f for f in FEATURES if f in df.columns]

# Fase 1 – conjuntos para calibración con early stopping
mask_it = (df['Ano'] < 2022) | ((df['Ano'] == 2022) & (df['MesNum'] <= 6))
inner_train = df[mask_it].copy() # 2012 – Jun 2022
inner_val = df[(df['Ano'] == 2022) & (df['MesNum'] >= 7)].copy() # Jul – Dic 2022

# Fase 2 – conjunto de entrenamiento final
full_train = df[df['Ano'] <= 2022].copy() # 2012 – 2022 completo

# Conjuntos de validación (nunca vistos durante el entrenamiento)
val23 = df[df['Ano'] == 2023].copy()
val24 = df[df['Ano'] == 2024].copy()
val25 = df[df['Ano'] == 2025].copy()

# Matrices de features y targets en escala Logarítmica
X_it = inner_train[FEATURES]; y_it = np.log1p(inner_train[TARGET])
X_iv = inner_val[FEATURES]; y_iv = np.log1p(inner_val[TARGET])
X_ft = full_train[FEATURES]; y_ft = np.log1p(full_train[TARGET])

# Verificaciones de integridad temporal
assert inner_train['Ano'].max() <= 2022, 'Error: inner_train contiene 2023+'
assert full_train['Ano'].max() == 2022, 'Error: full_train contiene 2023+'
assert val23['Ano'].min() > full_train['Ano'].max(), 'Error: overlap train-val23'

print(f'Variables del modelo : {len(FEATURES)}')
print(f'\nFase 1 – inner train (2012-Jun22): {len(inner_train):,} filas')
print(f'Fase 1 – inner val (Jul-Dic22): {len(inner_val):,} filas')
print(f'Fase 2 – full train (2012-2022): {len(full_train):,} filas')
print(f'\nValidación 2023 (1 año) : {len(val23):,} filas')
print(f'Validación 2024 (2 años): {len(val24):,} filas')
print(f'Validación 2025 (3 años): {len(val25):,} filas')
print(f'\nIntegridad temporal verificada: sin filtración de datos.')

```

Variables del modelo : 22

Fase 1 – inner train (2012-Jun22): 252,741 filas

Fase 1 – inner val (Jul-Dic22): 15,058 filas

Fase 2 – full train (2012-2022): 267,799 filas

Validación 2023 (1 año) : 31,998 filas

Validación 2024 (2 años): 35,460 filas

Validación 2025 (3 años): 35,754 filas

Integridad temporal verificada: sin filtración de datos.

5. Fase 1 — Calibración con early stopping

El modelo entrena sobre 2012–junio 2022 y monitorea su error en julio–diciembre 2022. Cuando el RMSE en ese conjunto deja de mejorar durante 200 rondas consecutivas, el entrenamiento se detiene y se registra el número de árbol que tuvo el menor error como `BEST_N`.

Parámetros principales:

- `max_depth=9` : profundidad máxima de cada árbol. A mayor profundidad, más capacidad de aprender pero también más riesgo de sobreajuste.
- `learning_rate=0.005` : tamaño del paso en cada iteración. Un valor pequeño requiere más árboles pero produce predicciones más estables y precisas.
- `subsample=0.8` y `colsample_bytree=0.8` : cada árbol ve el 80% de los datos y el 80% de las variables, lo que introduce diversidad y reduce el sobreajuste.
- `reg_lambda=1.5` : penalización L2 que evita que los pesos de las hojas sean demasiado grandes.

```
In [5]: PARAMS = dict(
    max_depth      = 9,
    learning_rate   = 0.005,
    subsample       = 0.8,
    colsample_bytree = 0.8,
    min_child_weight = 10,
    gamma           = 0.05,
    reg_alpha        = 0.1,
    reg_lambda       = 1.5,
    tree_method      = 'hist',
    device           = DEVICE,
    random_state     = 42
)

print(f'Fase 1 – calibración temporal ({DEVICE.upper()})...')
print(f'  Train: 2012-Jun 2022 | Val early stopping: Jul-Dic 2022')
t0 = time.time()

xgb_f1 = xgb.XGBRegressor(
    **PARAMS,
    n_estimators      = 20000,  # Límite máximo de árboles
    early_stopping_rounds = 200,  # parar si no mejora en 200 rondas
    eval_metric        = 'rmse'
)
xgb_f1.fit(
    X_it, y_it,
    eval_set=[(X_iv, y_iv)],
    verbose=2000  # mostrar progreso cada 2000 iteraciones
)

BEST_N = xgb_f1.best_iteration
print(f'\nFase 1 completada en {time.time()-t0:.1f}s')
print(f'Número óptimo de árboles: {BEST_N}')
print(f'RMSE mínimo en val      : {xgb_f1.best_score:.5f} (escala log)')
```

Fase 1 – calibración temporal (CUDA)...

Train: 2012-Jun 2022 | Val early stopping: Jul-Dic 2022

[0] validation_0-rmse:2.04543

[2000] validation_0-rmse:0.50062

[3235] validation_0-rmse:0.49969

Fase 1 completada en 24.9s

Número óptimo de árboles: 3035

RMSE mínimo en val : 0.49963 (escala log)

6. Fase 2 — Modelo final sobre 2012-2022 completo

Se entrena el modelo definitivo con el conjunto de datos completo de 2012–2022 y exactamente `BEST_N` árboles — el número que minimizó el error en la Fase 1.

Usar 2022 completo (en lugar de solo hasta junio) añade unos 31,000 registros adicionales al entrenamiento, lo que mejora la representación de los aeropuertos más pequeños.

```
In [6]: print(f'Fase 2 – entrenamiento final...')
print(f'  {BEST_N} árboles | 2012–2022 completo ({len(full_train):,} filas)')
t0 = time.time()

xgb_final = xgb.XGBRegressor(**PARAMS, n_estimators=BEST_N)
xgb_final.fit(X_ft, y_ft)

TIEMPO = round(time.time() - t0, 1)
print(f'Completado en {TIEMPO}s')
```

```
Fase 2 – entrenamiento final...
 3035 árboles | 2012–2022 completo (267,799 filas)
Completado en 21.1s
```

7. Evaluación con múltiples umbrales

La función `predecir` aplica la transformación inversa del logaritmo (`expm1`) para obtener el número de turistas en escala real.

La función `evaluar` calcula cinco métricas y las reporta para cada uno de los cuatro umbrales de negocio definidos al inicio. El número `n` entre paréntesis indica cuántos registros superan ese umbral — a medida que el umbral sube, quedan menos registros pero son los más relevantes para el negocio.

```
In [8]: def predecir(X):
        """Predice en número de turistas reales (revierte la transformación log)."""
        return np.maximum(np.expm1(xgb_final.predict(X)), 0)

def evaluar(y_real, pred, nombre, umbrales=UMBRALES):
    """Calcula y reporta todas las métricas de evaluación."""
    mae = mean_absolute_error(y_real, pred)
    rmse = np.sqrt(mean_squared_error(y_real, pred))
    r2 = r2_score(y_real, pred)
    mape_g = np.median(np.abs((y_real - pred) / np.maximum(y_real, 1)) * 100)

    print(f"\n{'='*62}")
    print(f'  {nombre}')
    print(f"{'='*62}")
    print(f'   R²           = {r2:.4f}')
    print(f'   MAE          = {mae:>10,.1f} turistas')
    print(f'   RMSE         = {rmse:>10,.1f} turistas')
    print(f'   MAPE global  = {mape_g:.1f}% (todos los registros, incl. segmentos pequeños)')

    mapes = {}
    for u in umbrales:
        mask = y_real >= u
        n = mask.sum()
        if n > 0:
            m = np.median(np.abs((y_real[mask] - pred[mask]) / np.maximum(y_real[mask], 1)) * 100)
            estado = 'DENTRO ±15%' if m <= 15 else f'a {m-15:.1f} pts del límite'
            print(f'   MAPE ≥{u:>7,} = {m:5.1f}% (n={n:,} registros) {estado}')
```

```

        mapes[u] = round(m, 1)
    else:
        print(f' MAPE ≥{u:>7,} = sin registros suficientes')

    return {'r2': round(r2,4), 'mae': round(mae,1), 'rmse': round(rmse,1),
            'mape_g': round(mape_g,1), 'mapes': mapes}

pred23 = predecir(val23[FEATURES])
pred24 = predecir(val24[FEATURES])
pred25 = predecir(val25[FEATURES])

m23 = evaluar(val23[TARGET].values, pred23, 'VALIDACIÓN 2023 – 1 año de distancia')
m24 = evaluar(val24[TARGET].values, pred24, 'VALIDACIÓN 2024 – 2 años de distancia')
m25 = evaluar(val25[TARGET].values, pred25, 'VALIDACIÓN 2025 – 3 años de distancia')

```

```

=====
VALIDACIÓN 2023 – 1 año de distancia
=====
R²                = 0.9726
MAE                =      103.2 turistas
RMSE               =      1,151.4 turistas
MAPE global       = 36.9% (todos los registros, incl. segmentos pequeños)
MAPE ≥ 250        = 16.5% (n=2,997 registros) a 1.5 pts del límite
MAPE ≥ 1,000      = 15.0% (n=1,589 registros) DENTRO ±15%
MAPE ≥ 10,000     = 15.7% (n=273 registros) a 0.7 pts del límite
MAPE ≥100,000     = 9.2% (n=29 registros) DENTRO ±15%

```

```

=====
VALIDACIÓN 2024 – 2 años de distancia
=====
R²                = 0.9655
MAE                =      105.1 turistas
RMSE               =      1,332.0 turistas
MAPE global       = 37.3% (todos los registros, incl. segmentos pequeños)
MAPE ≥ 250        = 15.5% (n=3,055 registros) a 0.5 pts del límite
MAPE ≥ 1,000      = 13.9% (n=1,589 registros) DENTRO ±15%
MAPE ≥ 10,000     = 16.8% (n=270 registros) a 1.8 pts del límite
MAPE ≥100,000     = 14.5% (n=35 registros) DENTRO ±15%

```

```

=====
VALIDACIÓN 2025 – 3 años de distancia
=====
R²                = 0.9620
MAE                =      109.2 turistas
RMSE               =      1,418.4 turistas
MAPE global       = 36.9% (todos los registros, incl. segmentos pequeños)
MAPE ≥ 250        = 15.6% (n=2,994 registros) a 0.6 pts del límite
MAPE ≥ 1,000      = 13.7% (n=1,586 registros) DENTRO ±15%
MAPE ≥ 10,000     = 16.7% (n=265 registros) a 1.7 pts del límite
MAPE ≥100,000     = 13.2% (n=38 registros) DENTRO ±15%

```

8. Importancia de variables

XGBoost registra cuánto contribuye cada variable a reducir el error a lo largo de todos los árboles del modelo. La importancia reportada es la ganancia promedio: cuánto mejora el error de predicción cuando se usa esa variable para dividir un nodo del árbol.

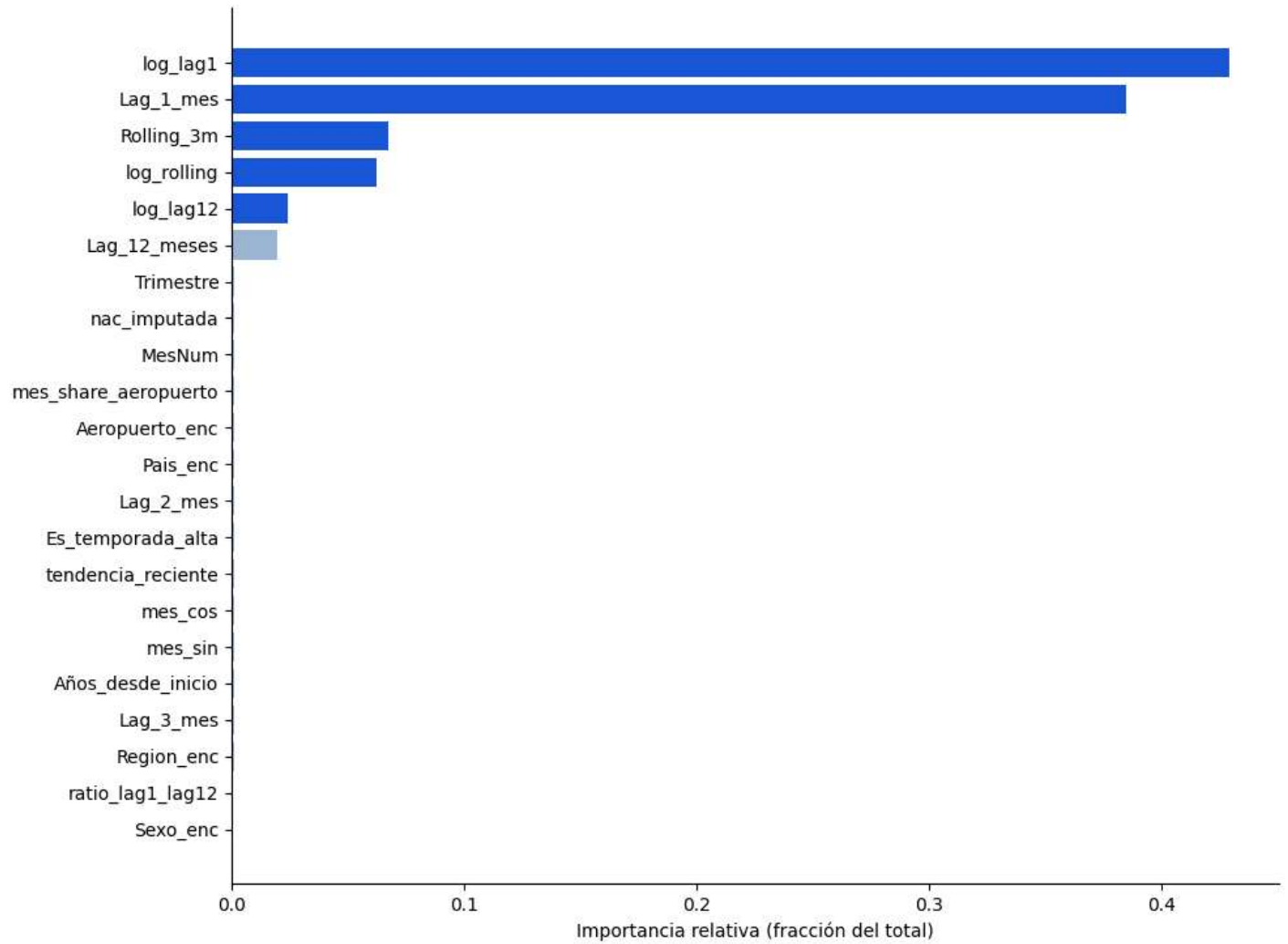
Las variables de rezago dominan porque el turismo del mes pasado es el predictor más fuerte del turismo del mes siguiente. Los rezagos logarítmicos (`log_lag1` , `log_rolling`) suelen tener alta importancia porque están en la misma escala que el target.

```
In [9]: importancias = pd.Series(
        xgb_final.feature_importances_, index=FEATURES
    ).sort_values(ascending=False)

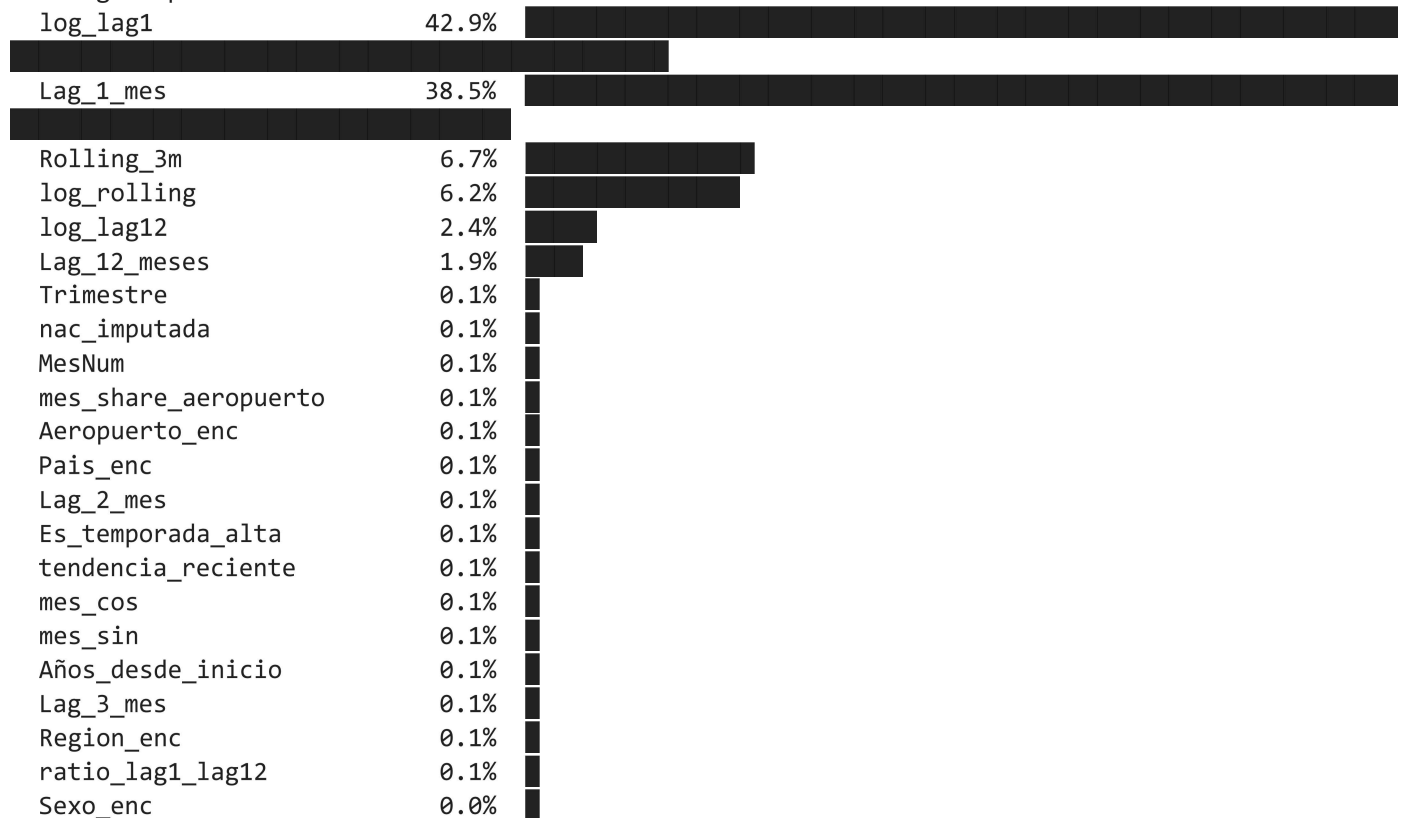
fig, ax = plt.subplots(figsize=(10, 8))
colores = ['#1d58d8' if i < 5 else '#9db4d6' for i in range(len(importancias))]
ax.barh(importancias.index[::-1], importancias.values[::-1],
        color=colores[::-1], edgecolor='none')
ax.set_title(
    'Importancia de variables - XGBoost\n(azul oscuro = top 5 más influyentes)',
    fontsize=13, pad=15
)
ax.set_xlabel('Importancia relativa (fracción del total)')
plt.tight_layout()
plt.savefig('xgb_importancia.png', dpi=150, bbox_inches='tight')
plt.show()

print('Ranking completo de variables:')
for f, v in importancias.items():
    barra = '█' * max(1, int(v * 250))
    print(f' {f:<24} {v*100:5.1f}% {barra}')
print(f'\nTop 5 acumulan: {importancias.head(5).sum()*100:.1f}% de la importancia total')
```

Importancia de variables — XGBoost
(azul oscuro = top 5 más influyentes)



Ranking completo de variables:



Top 5 acumulan: 96.8% de la importancia total

9. Diagnóstico visual

Gráfica izquierda — Dispersión Real vs Predicho (2024): Cada punto es un registro del año de validación 2024. La concentración cerca de la línea roja indica buena precisión. La dispersión visible en los valores bajos corresponde a los segmentos pequeños donde el error porcentual es alto pero el error absoluto es de pocos turistas.

Gráfica derecha — Error % por mes (2024, registros ≥ 250 turistas): La intensidad del azul indica la magnitud del error: azul más oscuro = mayor error en ese mes. La línea gris punteada es la referencia del 15%. La línea verde muestra el promedio anual.

```
In [10]: fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Gráfica 1: dispersión Real vs Predicho
y24 = val24[TARGET].values
p99 = np.percentile(y24, 99) # se omite el 1% superior para que la escala sea legible
m99 = y24 <= p99
axes[0].scatter(y24[m99], pred24[m99], alpha=0.08, s=4, color='#1d58d8', rasterized=True)
axes[0].plot([0, p99], [0, p99], 'r--', lw=2, label='Predicción perfecta')
axes[0].set_xlabel('Turistas reales')
axes[0].set_ylabel('Turistas predichos')
axes[0].set_title(f'Real vs Predicho - 2024\nR²={m24["r2"]} MAE={m24["mae"]:.0f} turistas')
axes[0].legend()

# Gráfica 2: error mensual
v24 = val24.copy()
v24['pred'] = pred24
v24['e_pct'] = abs(v24[TARGET] - v24['pred']) / v24[TARGET].clip(lower=1) * 100
```



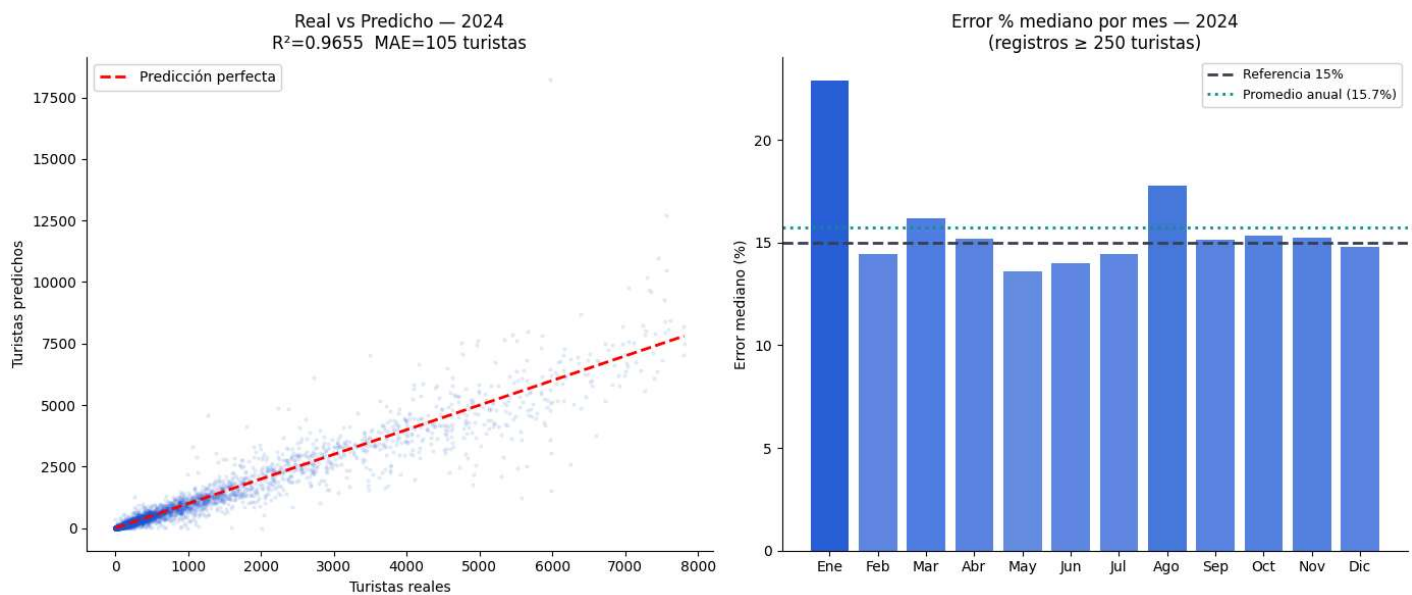
```

em = v24[v24[TARGET] >= 250].groupby('MesNum')['e_pct'].median()

intens = [0.3 + 0.65*(v/em.max()) for v in em.values]
cols_mes = [[29/255, 88/255, 216/255, a] for a in intens]
axes[1].bar(MESES, em.values, color=cols_mes, edgecolor='none')
axes[1].axhline(y=15, color='#374151', ls='--', lw=2, label='Referencia 15%')
axes[1].axhline(y=em.mean(), color='#0d9488', ls=':', lw=2,
                label=f'Promedio anual ({em.mean():.1f}%)')
axes[1].set_title('Error % mediano por mes — 2024\n(registros ≥ 250 turistas)')
axes[1].set_ylabel('Error mediano (%)')
axes[1].legend(fontsize=9)

plt.tight_layout()
plt.savefig('xgb_diagnostico.png', dpi=150, bbox_inches='tight')
plt.show()

```



10. Demostración — Cancún · EE.UU. · Mujeres

Comparación mes a mes entre lo que realmente llegó y lo que predijo el modelo, para el segmento más importante del dataset: mujeres residentes en Estados Unidos que arriban a Cancún.

Este segmento supera los 200,000 turistas en los meses pico y es el principal indicador de ocupación para las cadenas hoteleras de la Riviera Maya.

Las líneas verticales punteadas marcan el cambio de año. La franja azul sombreada muestra la magnitud del error mes a mes.

```

In [11]: val_todo = pd.concat([val23, val24, val25]).copy()
pred_todo = np.concatenate([pred23, pred24, pred25])
pred_s = pd.Series(pred_todo, index=val_todo.index)
val_todo['Prediccion'] = pred_s
val_todo['Error_pct'] = (
    abs(val_todo[TARGET] - val_todo['Prediccion']) / val_todo[TARGET] * 100
).round(1)
val_todo['Mes'] = (
    val_todo['MesNum'].map(dict(enumerate(MESES, 1))) +
    ' ' + val_todo['Ano'].astype(str).str[-2:]
)

```

```

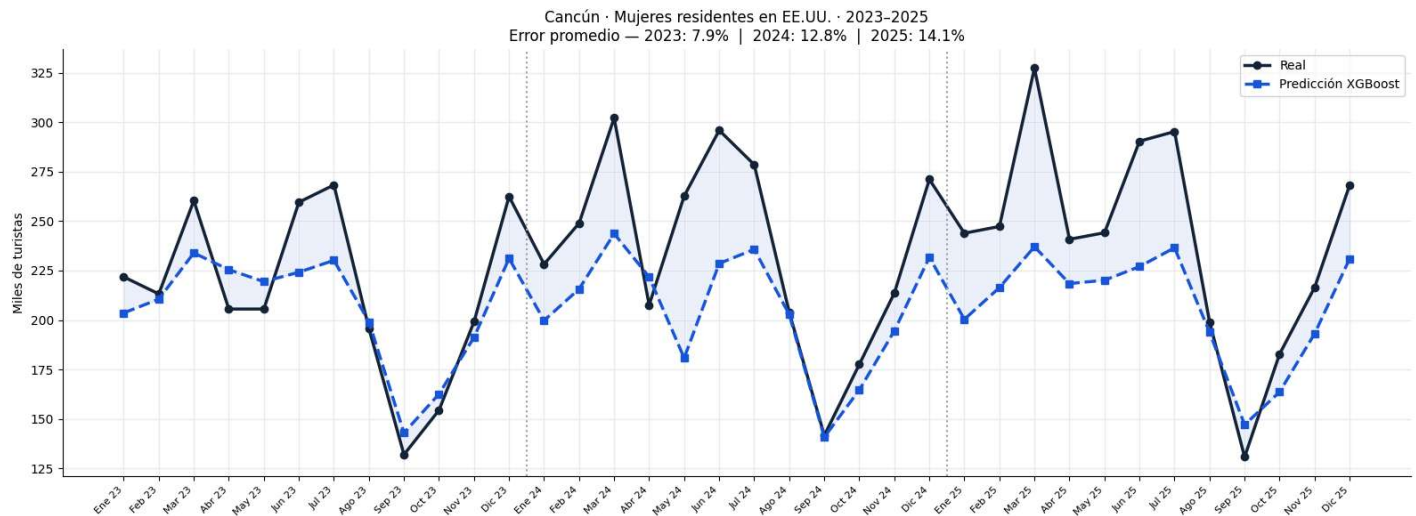
cancun = val_todo[
    (val_todo['Aeropuerto'] == 'cancun, q. roo') &
    (val_todo['Pais'] == 'estados unidos') &
    (val_todo['Sexo'] == 'mujer')
].copy().sort_values(['Ano', 'MesNum'])

err_anio = cancun.groupby('Ano')['Error_pct'].mean()
err_str = ' | '.join([f'{a}: {v:.1f}% ' for a, v in err_anio.items()])

fig, ax = plt.subplots(figsize=(16, 6))
x = range(len(cancun))
ax.plot(x, cancun[TARGET]/1000, 'o-', color='#14243a', lw=2.5, label='Real', ms=6)
ax.plot(x, cancun['Prediccion']/1000, 's--', color='#1d58d8', lw=2.5,
        label='Predicción XGBoost', ms=6)
ax.fill_between(x, cancun[TARGET]/1000, cancun['Prediccion']/1000,
                alpha=0.08, color='#1d58d8')
for cx in [12, 24]:
    ax.axvline(x=cx-0.5, color='#9ca3af', ls=':', lw=1.5)
ax.set_xticks(x)
ax.set_xticklabels(cancun['Mes'], rotation=45, ha='right', fontsize=8)
ax.set_ylabel('Miles de turistas')
ax.set_title(
    f'Cancún · Mujeres residentes en EE.UU. · 2023-2025\n'
    f'Error promedio - {err_str}',
    fontsize=12
)
ax.legend()
ax.grid(True, alpha=0.2)
plt.tight_layout()
plt.savefig('xgb_demo_cancun.png', dpi=150, bbox_inches='tight')
plt.show()

print('Detalle mes a mes:')
print(cancun[['Mes', TARGET, 'Prediccion', 'Error_pct']].to_string(index=False))

```



Detalle mes a mes:

Mes	Valor_Residencia	Prediccion	Error_pct
Ene 23	221806	203598.546875	8.2
Feb 23	213190	210559.093750	1.2
Mar 23	260371	233899.890625	10.2
Abr 23	205558	225472.359375	9.7
May 23	205572	219404.203125	6.7
Jun 23	259565	224167.062500	13.6
Jul 23	268271	230180.937500	14.2
Ago 23	195684	199111.984375	1.8
Sep 23	131970	142947.984375	8.3
Oct 23	154433	162615.703125	5.3
Nov 23	199131	191252.500000	4.0
Dic 23	262450	231116.015625	11.9
Ene 24	228242	199751.234375	12.5
Feb 24	248937	215696.328125	13.4
Mar 24	302152	243651.125000	19.4
Abr 24	207580	221750.328125	6.8
May 24	262677	180783.765625	31.2
Jun 24	295894	228579.218750	22.7
Jul 24	278541	235743.625000	15.4
Ago 24	203933	202798.203125	0.6
Sep 24	141668	140783.843750	0.6
Oct 24	177578	164676.687500	7.3
Nov 24	213618	194519.390625	8.9
Dic 24	271111	231723.593750	14.5
Ene 25	243891	200365.765625	17.8
Feb 25	247294	216392.125000	12.5
Mar 25	327314	237069.968750	27.6
Abr 25	240831	218434.843750	9.3
May 25	244125	220116.781250	9.8
Jun 25	290404	227018.093750	21.8
Jul 25	295262	236522.343750	19.9
Ago 25	199061	193755.671875	2.7
Sep 25	130766	147091.093750	12.5
Oct 25	182875	163673.703125	10.5
Nov 25	216509	193149.062500	10.8
Dic 25	268268	231013.093750	13.9

11. Tabla de resultados consolidada

Resumen de todas las métricas para los tres años de validación. El patrón esperado es que la precisión disminuya levemente al alejarse del período de entrenamiento — si 2025 tuviera exactamente el mismo error que 2023, el modelo podría estar sobreajustado a un período específico.

```
In [12]: print('=' * 82)
print('RESULTADOS - XGBoost')
print(f'Entrenamiento: 2012-2022 | {len(full_train):,} observaciones | {len(FEATURES)} variables')
print(f'Árboles: {BEST_N} | lr: 0.005 | max_depth: 9 | {DEVICE.upper()}')
print('=' * 82)

header = f"{'Año de validación':<18} {'R²':>8} {'MAE':>8} {'MAPE global':>12}"
for u in UMBRALES:
    header += f" {'MAPE≥'+str(u):>13}"
print(header)
print('-' * 82)
```

```

for anio, m in [('2023 (1 año)', m23), ('2024 (2 años)', m24), ('2025 (3 años)', m25)]:
    row = f"{anio:<18} {m['r2']:>8} {m['mae']:>8,.1f} {m['mape_g']:>11.1f}%"
    for u in UMBRALES:
        v = m['mapes'].get(u, None)
        row += f" {(str(v)+'%') if v else 'n/a':>13}"
    print(row)

print('=' * 82)

```

=====

RESULTADOS – XGBoost

Entrenamiento: 2012-2022 | 267,799 observaciones | 22 variables

Árboles: 3035 | lr: 0.005 | max_depth: 9 | CUDA

=====

Año de validación	R ²	MAE	MAPE global	MAPE≥250	MAPE≥1000	MAPE≥10000	MAP
2023 (1 año)	0.9726	103.2	36.9%	16.5%	15.0%	15.7%	
9.2%							
2024 (2 años)	0.9655	105.1	37.3%	15.5%	13.9%	16.8%	
14.5%							
2025 (3 años)	0.962	109.2	36.9%	15.6%	13.7%	16.7%	
13.2%							

=====

12. Guardar modelo y predicciones

Se guardan los archivos necesarios para el análisis posterior y para cargar en el dashboard de visualización.

El archivo `predicciones_xgb.csv` es el más importante para el dashboard — contiene las predicciones mes a mes de 2023 a 2025 para todos los segmentos del modelo.

```

In [13]: from google.colab import files

xgb_final.save_model('modelo_xgboost.json')
with open('modelo_xgboost.pkl', 'wb') as f:
    pickle.dump(xgb_final, f)

pred_out = val_todo[[
    'Ano', 'MesNum', 'Aeropuerto', 'Pais', 'Sexo',
    TARGET, 'Prediccion', 'Error_pct'
]].copy().rename(columns={'Prediccion': 'Prediccion_XGB'})
pred_out.to_csv('predicciones_xgb.csv', index=False)

metricas = {
    'modelo': 'XGBoost', 'entrenamiento': '2012-2022',
    'best_n': BEST_N, 'device': DEVICE, 'tiempo_s': TIEMPO, 'features': FEATURES,
    'r2_2023': m23['r2'], 'mapes_2023': m23['mapes'],
    'r2_2024': m24['r2'], 'mapes_2024': m24['mapes'],
    'r2_2025': m25['r2'], 'mapes_2025': m25['mapes'],
}
with open('metricas_xgb.json', 'w') as f:
    json.dump(metricas, f, indent=2)

print('=' * 55)
print('ARCHIVOS GENERADOS')
print('=' * 55)
for a in ['modelo_xgboost.json', 'modelo_xgboost.pkl', 'predicciones_xgb.csv',

```

```

        'metricas_xgb.json', 'xgb_importancia.png', 'xgb_diagnostico.png', 'xgb_demo_cancun.png')
    if os.path.exists(a):
        kb = os.path.getsize(a) / 1024
        print(f' {a:<44} {kb/1024 if kb>1024 else kb:>6.1f} {"MB" if kb>1024 else "KB"}')
        files.download(a)

print(f'\nMAPE negocio ≥ 1,000 turistas:')
for anio, m in [('2023', m23), ('2024', m24), ('2025', m25)]:
    print(f' {anio}: {m["mapes"].get(1000, "n/a")}%')

```

```

=====
ARCHIVOS GENERADOS
=====

```

modelo_xgboost.json	100.1 MB
modelo_xgboost.pkl	62.6 MB
predicciones_xgb.csv	6.1 MB
metricas_xgb.json	0.9 KB
xgb_importancia.png	92.3 KB
xgb_diagnostico.png	145.4 KB
xgb_demo_cancun.png	224.2 KB

```

MAPE negocio ≥ 1,000 turistas:
2023: 15.0%
2024: 13.9%
2025: 13.7%

```